

project_summary.txt

```
----data
----lib
----src
|   platformio.ini
|   project_summary.txt
|   project_summary.txt.pdf
|   zvuk(refaktor).code-workspace
|   stations.txt
|   peфaкт.txt
----ESP32-vs1053_ext
----additional info
----examples
----src
|   library.properties
|   LICENSE
|   README.md
|   Breadboard.jpg
|   ESP32_dev_board.jpg
|   MP3_Board.gif
|   MP3_board_schematic.jpg
|   Webstreams.txt
----vs1053_ext
|   vs1053_ext.ino
|   vs1053b-patches-flac.h
|   vs1053_ext.cpp
|   vs1053_ext.h
----core
|   config.h
|   global_state.cpp
|   global_state.h
|   main.cpp
|   mqtt_module.cpp
|   mqtt_module.h
|   web_interface.cpp
|   web_interface.h
|   web_server.cpp
|   web_server.h
|   zvuk(S2 Mini).code-workspace
|   audio_playback.cpp
|   audio_playback.h
|   audio_recorder.cpp
|   audio_recorder.h
|   display_module.cpp
|   display_module.h
|   fs_module.cpp
|   fs_module.h
|   playlist_parser.cpp
|   playlist_parser.h
|   system_logger.cpp
|   system_logger.h

=== Файл: config.h ===
#pragma once
#include <Arduino.h>
#include <LittleFS.h>

// Настройки Wi-Fi (ИСПРАВЛЕНО: добавлен static против ошибок множественного определения)
static const char* WIFI_SSID = "Sloboda100";
static const char* WIFI_PASSWORD = "2716192023";

// Наша новая аппаратная схема пинов для Lolin S2 Mini
#define PIN_SPI_SCK 7
#define PIN_SPI_MISO 9
#define PIN_SPI_MOSI 11
#define VS1053_CS 12 // XCS
#define VS1053_DCS 14 // XDCS
#define VS1053_DREQ 13 // DREQ
#define VS1053_RST 18 // XRESET
#define I2C_SDA 33
#define I2C_SCL 35

// Структура для динамического списка станций
struct Station {
    String name;
    String url;
};

// Резервируем до 50 станций в памяти
const int MAX_STATIONS = 50;

// Указываем, что сами данные физически лежат в main.cpp
extern Station stationList[MAX_STATIONS];
extern int stationCount; // <--- СТРОГО С extern! Никаких локальных дубликатов!
```

```

// Переменные для хранения настроек звука
extern int currentVolume;
extern int currentBass;
extern int currentTreble;

// Функции для работы с файловой системой
void initFileSystem();
void loadPlaylist();

// --- НАСТРОЙКИ MQTT ДЛЯ ОФИЦИАЛЬНОЙ ИНТЕГРАЦИИ YORADIO ---
// (ИСПРАВЛЕНО: добавлен static против ошибок множественного определения)
static const char* MQTT_SERVER = "192.168.1.23";
const int MQTT_PORT = 1883;
static const char* MQTT_USER = "";
static const char* MQTT_PASSWORD = "";

// Жестко фиксируем базовый топик для интеграции автора yoradio
#define MQTT_CLIENT_ID "homeradio" // Имя вашего радио
#define MQTT_BASE_TOPIC "yoradio/homeradio"

=== Файл: global_state.cpp ===
#include "global_state.h"

// Физическое выделение памяти под переменные состояния
Station stationList[MAX_STATIONS];
int stationCount = 0;
int currentVolume = 12;
int currentBass = 0;
int currentTreble = 0;
int currentStationIdx = -1;
String currentStationName = "No Station";
String currentTrack = "No Title";
String customUrl = "";
bool changeStationFlag = false;
bool isRecording = false;

=== Файл: global_state.h ===
#pragma once
#include "config.h"

// Объявление разделяемых переменных состояния для всех модулей
extern Station stationList[MAX_STATIONS];
extern int stationCount;
extern int currentVolume;
extern int currentBass;
extern int currentTreble;
extern int currentStationIdx;
extern String currentStationName;
extern String currentTrack;
extern String customUrl;
extern bool changeStationFlag;
extern bool isRecording;

=== Файл: main.cpp ===
#include <Arduino.h>
#include <WiFi.h>
#include <Update.h>
#include <GyverDBFile.h>

// Подключение системного ватчдога ESP32
#include "esp_task_wdt.h"

#include "config.h"
#include "global_state.h"
#include "web_server.h"
#include "mqtt_module.h"

// Подключаем модули из подпапки core
#include "core/fs_module.h"
#include "core/playlist_parser.h"
#include "core/audio_playback.h"
#include "core/audio_recorder.h"
#include "core/display_module.h"
#include "core/system_logger.h"

unsigned long lastWiFiCheck = 0;
const unsigned long wifiCheckInterval = 10000;
unsigned long lastDisplayUpdate = 0;
unsigned long lastHealthCheck = 0;

void setup() {
    Serial.begin(115200);
    delay(2000);

```

```

RESET_REASON r0 = rtc_get_reset_reason(0);
sysLog("warn:", "АППАРАТ", "Причина прошлого сброса CPU: " + getResetReason(r0));
sysLog("info:", "СТАРТ", "Запуск радио. Инициализация периферии...");

// ИСПРАВЛЕНИЕ: Мы полностью убрали esp_task_wdt_init и esp_task_wdt_add,
// чтобы сторожевой таймер не перезагружал плату во время блокирующего OTA-апдейта!

initDisplay();
initFileSystem();
loadPlaylist();
initAudio();

WiFi.mode(WIFI_MODE_STA);
WiFi.setAutoReconnect(true);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
sysLog("info:", "СЕТЬ", "Подключение к Wi-Fi: " + String(WIFI_SSID));

unsigned long startAttempt = millis();
while (WiFi.status() != WL_CONNECTED && millis() - startAttempt < 10000) {
    delay(500);
}

if (WiFi.status() == WL_CONNECTED) {
    sysLog("info:", "СЕТЬ", "Успешное подключение. IP: " + WiFi.localIP().toString());
    configTime(3600 * 3, 0, "pool.ntp.org", "ru.pool.ntp.org");
} else {
    sysLog("err:", "СЕТЬ", "Ошибка Wi-Fi! Тайм-аут 10 секунд.");
}

initWebServer();
initMQTT();
logSystemHealth();

if (db.has(SH("st_id"))) {
    int savedStationIdx = db[SH("st_id")].toInt();
    if (savedStationIdx >= 0 && savedStationIdx < stationCount) {
        currentStationIdx = savedStationIdx;
        currentStationName = stationList[currentStationIdx].name;
        sysLog("info:", "СТАРТ", "Восстановлена станция: " + currentStationName);
        updateDisplay(currentStationName.c_str(), "Connecting...");
        startRadioStream(stationList[currentStationIdx].url.c_str());
    }
}
}

void loop() {
    // Вызов обработчика веб-запросов (при OTA сервер заблокирует код внутри этой функции)
    handleWebRequests();

    if (millis() - lastWiFiCheck > wifiCheckInterval) {
        lastWiFiCheck = millis();
        if (WiFi.status() != WL_CONNECTED) {
            sysLog("warn:", "СЕТЬ", "Связь потеряна! Переподключение...");
            WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
        }
    }

    if (millis() - lastHealthCheck > 60000) {
        lastHealthCheck = millis();
        if (WiFi.status() == WL_CONNECTED) logSystemHealth();
    }

    if (WiFi.status() == WL_CONNECTED) loopMQTT();

    loopAudioPlayback();
    loopAudioRecorder();

    if (changeStationFlag) {
        changeStationFlag = false;
        if (currentStationIdx >= 0 && currentStationIdx < stationCount) {
            String url = stationList[currentStationIdx].url;
            updateDisplay(stationList[currentStationIdx].name.c_str(), "Checking...");
            if (checkUrlAvailability(url)) {
                updateDisplay(stationList[currentStationIdx].name.c_str(), "Connecting...");
                startRadioStream(url.c_str());
                updateDisplay(stationList[currentStationIdx].name.c_str(), "PLAYING");
                currentStationName = stationList[currentStationIdx].name;
            } else {
                updateDisplay(stationList[currentStationIdx].name.c_str(), "HOST ERROR");
            }
        }
    }

    if (millis() - lastDisplayUpdate > 1000) {
        lastDisplayUpdate = millis();
        updateDisplay(currentStationName.c_str(), currentTrack.c_str());
    }
}

```

```

    }
}

=== Файл: mqtt_module.cpp ===
#include "mqtt_module.h"
#include "config.h"
#include "global_state.h"
#include "core/system_logger.h"
#include "core/audio_playback.h"
#include <WiFi.h>

// Физическое выделение памяти под сетевые клиенты
WiFiClient espClient;
PubSubClient mqttClient(espClient);

const char *topic_command = MQTT_BASE_TOPIC "/command";
const char *topic_status = MQTT_BASE_TOPIC "/status";
const char *topic_playlist = MQTT_BASE_TOPIC "/playlist";
const char *topic_volume = MQTT_BASE_TOPIC "/volume";

String currentMqttTrack = "Radio Ready";
String currentMqttStatus = "playing";

void mqttPublishPlaylist() {
    if (!mqttClient.connected()) return;
    // Возвращен оригинальный путь получения файла через встроенный веб-сервер
    String playlistUrl = "http://" + WiFi.localIP().toString() + "/fetch?path=/ha_playlist.txt";
    mqttClient.publish(topic_playlist, playlistUrl.c_str(), true);
}

void mqttPublishStatus() {
    if (!mqttClient.connected()) return;

    int haVolume = (int)((currentVolume * 254) / 21);
    haVolume = constrain(haVolume, 0, 254);
    mqttClient.publish(topic_volume, String(haVolume).c_str(), true);

    String track = currentMqttTrack;
    track.replace("\\", "");
    if (track.length() == 0) track = "Radio Ready";

    String station = "";
    if (currentStationIdx >= 0 && currentStationIdx < stationCount) {
        station = stationList[currentStationIdx].name;
    } else {
        station = currentStationName;
    }
    station.replace("\\", "");
    if (station.length() == 0 || station == "No Station") station = "Sloboda Radio";

    int currentIdx = (currentStationIdx >= 0) ? currentStationIdx + 1 : 1;
    int isStatusPlaying = (currentMqttStatus == "playing") ? 1 : 0;

    // Сборка оригинального JSON YoRadio: строки "title" и "artist"
    String json = "{";
    json += "\"title\": \"" + track + "\", ";
    json += "\"artist\": \"" + station + "\", ";
    json += "\"name\": \"" + station + "\", ";
    json += "\"on\": 1, ";
    json += "\"status\": " + String(isStatusPlaying) + ", ";
    json += "\"station\": " + String(currentIdx);
    json += "}";

    mqttClient.publish(topic_status, json.c_str(), true);
}

void mqttCallback(char *topic, byte *payload, unsigned int length) {
    String body = "";
    for (unsigned int i = 0; i < length; i++) body += (char)payload[i];
    body.trim();

    syslog("info:", "MQTT", "Получена команда от Home Assistant: -> " + body);

    if (body.startsWith("vol ")) {
        int haVol = body.substring(4).toInt();
        updateVolume((haVol * 21) / 254);
    } else if (body.startsWith("play ")) {
        int targetIdx = body.substring(5).toInt() - 1;
        if (targetIdx >= 0 && targetIdx < stationCount) {
            currentStationIdx = targetIdx;
            customUrl = "";
            changeStationFlag = true;
            currentMqttStatus = "playing";
        }
    } else if (body == "next" && stationCount > 0) {
        currentStationIdx = (currentStationIdx + 1) % stationCount;
    }
}

```

```

        customUrl = "";
        changeStationFlag = true;
        currentMqttStatus = "playing";
    } else if (body == "prev" && stationCount > 0) {
        currentStationIdx = (currentStationIdx - 1 + stationCount) % stationCount;
        customUrl = "";
        changeStationFlag = true;
        currentMqttStatus = "playing";
    } else if (body == "stop") {
        currentMqttStatus = "stopped";
        changeStationFlag = false;
        if (player != nullptr) player->stop_mp3client();
    } else if (body == "start" || body == "play") {
        currentMqttStatus = "playing";
        changeStationFlag = true;
    } else if (body == "turnoff") {
        currentMqttStatus = "stopped";
        changeStationFlag = false;
        if (player != nullptr) player->stop_mp3client();
    }
    mqttPublishStatus();
}

void connectMQTT() {
    if (!mqttClient.connected()) {
        Serial.print("[MQTT]: Подключение к брокеру...");
        if (mqttClient.connect(MQTT_CLIENT_ID, MQTT_USER, MQTT_PASSWORD)) {
            Serial.println(" успешно!");
            mqttClient.subscribe(topic_command);
            mqttPublishPlaylist();
            mqttPublishStatus();
        } else {
            Serial.printf(" ошибка, rc=%d. Повтор через 5 сек.\n", mqttClient.state());
        }
    }
}

void initMQTT() {
    mqttClient.setServer(MQTT_SERVER, MQTT_PORT);
    mqttClient.setCallback(mqttCallback);
}

void loopMQTT() {
    if (!mqttClient.connected()) {
        static unsigned long lastReconnect = 0;
        if (millis() - lastReconnect > 5000) {
            lastReconnect = millis();
            connectMQTT();
        }
    }
    mqttClient.loop();

    static unsigned long lastStatusTime = 0;
    if (millis() - lastStatusTime > 10000) {
        lastStatusTime = millis();
        mqttPublishStatus();
    }
}

void mqttPublishTrack(const char *trackInfo) {
    currentMqttTrack = String(trackInfo);
    currentMqttStatus = "playing";
    mqttPublishStatus();
}

=== Файл: mqtt_module.h ===
#include <Arduino.h>
#include <WiFi.h> // Добавлен для распознавания типа WiFiClient
#include <PubSubClient.h>

// Экспортируем сетевые клиенты для возможности диагностики
extern WiFiClient espClient;
extern PubSubClient mqttClient;

void initMQTT();
void loopMQTT();
void mqttPublishTrack(const char *trackInfo);
void mqttPublishStatus();
void mqttPublishPlaylist();

=== Файл: web_interface.cpp ===
#include "web_interface.h"
#include "config.h"
#include "global_state.h"
#include "web_server.h"

```

```

// Подключаем модули из подпапки core
#include "core/system_logger.h"
#include "core/audio_playback.h"
#include "core/audio_recorder.h"
#include "core/fs_module.h"
#include <Update.h>

// Регистрация ключей БД по спецификации (стр. 14 документации)
DB_KEYS(kk, vol, bass, treble, st_id, c_url, txt_st, txt_tr, sys_log);

String getStationsOptions()
{
    if (stationCount == 0)
        return "Нет станций";
    String list = "";
    for (int i = 0; i < stationCount; i++)
    {
        list += stationList[i].name;
        if (i < stationCount - 1)
            list += ";"; // Разделитель строго ';' (стр. 8)
    }
    return list;
}

void buildInterface(sets::Builder &b)
{
    {
        sets::Group g(b, "Текущий поток"); // стр. 31 документации
        b.Label(kk::txt_st, "Станция", currentStationName);
        b.Label(kk::txt_tr, "Трек", currentTrack);

        if (b.Slider(kk::vol, "Громкость", 0, 21, 1))
        {
            // Использование макроса ключей для чтения (стр. 1, 14 документации)
            extern GyverDBFile db;
            updateVolume(db[kk::vol].toInt());
        }

        {
            sets::Buttons btns(b); // Контейнер кнопок (стр. 33 документации)
            if (b.Button("⏮ Назад"))
            {
                if (stationCount > 0)
                {
                    extern GyverDBFile db;
                    currentStationIdx = (currentStationIdx - 1 + stationCount) % stationCount;
                    db[kk::st_id] = currentStationIdx;
                    db.update();
                    customUrl = "";
                    changeStationFlag = true;
                }
            }
            if (b.Button("▶ Плей"))
            {
                if (currentStationIdx >= 0 && currentStationIdx < stationCount)
                    changeStationFlag = true;
            }
            if (b.Button("■ Стоп"))
            {
                changeStationFlag = false;
                if (player != nullptr)
                    player->stop_mp3client();
            }
            if (b.Button("⏭ Вперед"))
            {
                if (stationCount > 0)
                {
                    extern GyverDBFile db;
                    currentStationIdx = (currentStationIdx + 1) % stationCount;
                    db[kk::st_id] = currentStationIdx;
                    db.update();
                    customUrl = "";
                    changeStationFlag = true;
                }
            }
        }
    }
}

{
    sets::Group g(b, "Настройки тембра");
    bool toneChanged = false;
    if (b.Slider(kk::bass, "Усиление НЧ (Бас)", 0, 15, 1))
        toneChanged = true;
    if (b.Slider(kk::treble, "Усиление ВЧ (Требл)", -8, 7, 1))
        toneChanged = true;
    if (toneChanged)
    {

```

```

        extern GyverDBFile db;
        updateTone(db[kk::bass].toInt(), db[kk::treble].toInt());
    }
}
{
    sets::Group g(b, "Выбор источника");
    if (b.Select(kk::st_id, "Выбрать станцию", getStationsOptions()))
    {
        extern GyverDBFile db;
        currentStationIdx = db[kk::st_id].toInt();
        customUrl = "";
        changeStationFlag = true;
    }
    if (b.Input(kk::c_url, "Кастомный URL"))
    {
        extern GyverDBFile db;
        customUrl = db[kk::c_url].toString();
        if (customUrl.length() > 5)
        {
            currentStationIdx = -1;
            changeStationFlag = true;
        }
    }
}
{
    sets::Group g(b, "Управление записью (VS1053)");
    sets::Buttons btns(b);
    if (b.Button("● Старт записи"))
        startRecording();
    if (b.Button("□ Стоп записи"))
        stopRecording();
}
{
    sets::Group g(b, "Диагностика и Логи");

    // Читаем лог-файл с диска для вывода в веб-интерфейс
    if (LittleFS.exists("/log.txt"))
    {
        File logFile = LittleFS.open("/log.txt", "r");
        if (logFile)
        {
            while (logFile.available())
            {
                webLogger.print(logFile.readStringUntil('\n'));
            }
            logFile.close();
        }
    }
    b.Log(kk::sys_log, webLogger, "Журнал событий");

    if (b.Button("Очистить историю логов"))
    {
        removeFile("/log.txt");
        syslog("info:", "СИСТЕМА", "История логов успешно очищена.");
        b.reload(); // Перезагрузка UI (стр. 2, 4 документации)
    }
}
}

void updateInterface(sets::Updater &upd)
{
    // ЖЕСТКАЯ ЗАЩИТА ОТА: Если МК занят прошивкой, отсекаем отправку AJAX ответов (стр. 20)
    if (Update.isRunning())
        return;

    upd.update(kk::txt_st, currentStationName);
    upd.update(kk::txt_tr, currentTrack);
    upd.update(kk::sys_log, webLogger); // Спецификация обновления лога (стр. 10 документации)
}

=== Файл: web_interface.h ===
#pragma once
#include <SettingsGyver.h>

// Объявление функций UI, которые будут вызываться ядром сервера
void buildInterface(sets::Builder &b);
void updateInterface(sets::Updater &upd);

=== Файл: web_server.cpp ===
#include "web_server.h"
#include "web_interface.h"
#include "core/system_logger.h" // Исправленный путь к папке core/
#include <LittleFS.h>

// Физическое выделение памяти под БД и сервер (стр. 13-14 документации)
GyverDBFile db(&LittleFS, "/db.bin");

```

```

SettingsGyver sett("ESP32S2 RefaktoRadio", &db);

void initWebServer() {
    if (LittleFS.begin(true)) {
        db.begin();

        // РЕГИСТРАЦИЯ И ИНИЦИАЛИЗАЦИЯ ДЕФОЛТНЫХ КЛЮЧЕЙ БД СТРОГО ПО СУЩЕСТВУЮЩЕМУ МАКРОСУ
        // Метод .init() создает ячейку только если её еще нет, не перезаписывая старые данные
        db.init(SH("vol"), 12);
        db.init(SH("bass"), 0);
        db.init(SH("treble"), 0);
        db.init(SH("st_id"), 0);
        db.init(SH("c_url"), "");
        db.update(); // Принудительно сбрасываем структуру на Flash

        syslog("info:", "СЕРВЕР", "База данных успешно проинициализирована начальными ключами.");
    } else {
        syslog("err:", "СЕРВЕР", "КРИТИКА: LittleFS недоступен, БД не запущена!");
    }

    sett.begin();
    sett.onBuild(buildInterface);
    sett.onUpdate(updateInterface);
    syslog("info:", "СЕРВЕР", "Веб-сервер успешно запущен.");
}

void handleWebRequests() {
    sett.tick(); // Вызывается в главном loop (стр. 3 документации)
}

=== Файл: web_server.h ===
#pragma once
#include <GyverDBFile.h>
#include <SettingsGyver.h>

// Экспортируем объекты для связи с интерфейсом
extern GyverDBFile db;
extern SettingsGyver sett;

void initWebServer();
void handleWebRequests();

=== Файл: zvuk(S2 Mini).code-workspace ===
{
    "folders": [
        {
            "path": ".."
        }
    ],
    "settings": {}
}

=== Файл: audio_playback.cpp ===
#include "audio_playback.h"
#include "../config.h"
#include "../global_state.h"
#include "../mqtt_module.h"
#include <SPI.h>

VS1053* player = nullptr;

// Чистая реализация системных перехватчиков библиотеки Wollie VS1053
void vs1053_showstreamtitle(const char *info) {
    if (info != nullptr) {
        currentTrack = String(info);
        currentTrack.trim();

        // Отправляем реальное название трека в брокер Home Assistant
        mqttPublishTrack(currentTrack.c_str());
    }
}

void vs1053_showstreaminfo(const char *info) {
    Serial.printf("[ПОТОК INFO]: %s\n", info);
}

void initAudio() {
    pinMode(VS1053_RST, OUTPUT);
    digitalWrite(VS1053_RST, LOW);
    delay(50);
    digitalWrite(VS1053_RST, HIGH);
    delay(50);

    player = new VS1053(VS1053_CS, VS1053_DCS, VS1053_DREQ, HSPI, PIN_SPI_MOSI, PIN_SPI_MISO, PIN_SPI_SCK);
    player->begin();
}

```



```

    player->setVolume(currentVolume);

    uint8_t toneData[] = { (uint8_t)currentTreble, 6, (uint8_t)currentBass, 5 };
    player->setTone(toneData);

    Serial.println("[Звук]: Модуль VS1053 успешно запущен.");
}

void startRadioStream(const char* url) {
    if (player != nullptr && !isRecording) {
        currentTrack = "Loading...";
        mqttPublishTrack(currentTrack.c_str());
        player->connecttohost(url);
        player->setVolume(currentVolume);
    }
}

void updateVolume(int vol) {
    if (player != nullptr) {
        currentVolume = constrain(vol, 0, 21);
        player->setVolume(currentVolume);
        Serial.printf("[Звук]: Новая громкость: %d\n", currentVolume);
    }
}

void updateTone(int bass, int treble) {
    if (player != nullptr) {
        currentBass = constrain(bass, 0, 15);
        currentTreble = constrain(treble, -8, 7);

        uint8_t toneData[] = { (uint8_t)currentTreble, 6, (uint8_t)currentBass, 5 };
        player->setTone(toneData);
        Serial.printf("[Звук]: Эквалайзер -> БАС: %d, ТРЕБЛ: %d\n", currentBass, currentTreble);
    }
}

void loopAudioPlayback() {
    if (player != nullptr && !isRecording) {
        player->loop();
    }
}

=== Файл: audio_playback.h ===
#pragma once
#include <Arduino.h>
#include "vs1053_ext.h"

extern VS1053* player;

void initAudio();
void startRadioStream(const char* url);
void updateVolume(int vol);
void updateTone(int bass, int treble);
void loopAudioPlayback();

// Чистые объявления функций без extern "C", строго соответствующие библиотеке
void vs1053_showstreamtitle(const char *info);
void vs1053_showstreaminfo(const char *info);

=== Файл: audio_recorder.cpp ===
#include "audio_recorder.h"
#include "../global_state.h"
#include "audio_playback.h"
#include "system_logger.h"
#include <LittleFS.h>

File recordFile;

void startRecording() {
    if (player == nullptr || isRecording) return;
    syslog("info:", "ЗАПИСЬ", "Запуск аппаратной записи звука...");

    // Метод stop в библиотеке Wollie вызывается напрямую, он сам проверяет состояние
    player->stop_mp3client();

    recordFile = LittleFS.open("/record.wav", "w");
    if (!recordFile) {
        syslog("err:", "ЗАПИСЬ", "Не удалось создать файл /record.wav");
        return;
    }
    isRecording = true;
}

void stopRecording() {
    if (!isRecording) return;
    isRecording = false;
}

```

```

    if (recordFile) {
        recordFile.close();
        syslog("info:", "ЗАПИСЬ", "Аудиозапись успешно сохранена в /record.wav");
    }
}

void loopAudioRecorder() {
    if (!isRecording || !recordFile) return;

    // Временная заглушка для компиляции. Чтение регистров VS1053 для оцифровки
    // будет реализовано через публичные методы плеера на этапе настройки битрейта.
    uint8_t dummyBuf[2] = {0, 0};
    recordFile.write(dummyBuf, 2);
}

=== Файл: audio_recorder.h ===
#pragma once

void startRecording();
void stopRecording();
void loopAudioRecorder();

=== Файл: display_module.cpp ===
#include "display_module.h"
#include "../config.h" // <--- С выходом наверх!
#include <Wire.h>
#include <WiFi.h>
#include <time.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void initDisplay() {
    Wire.begin(I2C_SDA, I2C_SCL);
    if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println(F("[Экран]: Ошибка SSD1306!"));
        return;
    }
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 20);
    display.println("--- INTERNET RADIO ---");
    display.display();
    Serial.println(F("[Экран]: Успешно запущен.));
}

String utf8rus(String source) {
    int i;
    String target = "";
    unsigned char a, b;
    for (i = 0; i < source.length(); i++) {
        a = source[i];
        if (a < 128) {
            target += (char)a;
        } else if (a == 208 || a == 209) {
            b = source[++i];
            if (a == 208 && b == 129) target += (char)161;
            if (a == 209 && b == 145) target += (char)177;
            if (a == 208 && b >= 144 && b <= 191) target += (char)(b + 48);
            if (a == 209 && b >= 128 && b <= 143) target += (char)(b + 112);
        }
    }
    return target;
}

void updateDisplay(const char* station, const char* track) {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);

    struct tm timeinfo;
    char timeStr[6] = "--:--";
    if (getLocalTime(&timeinfo)) {
        strftime(timeStr, sizeof(timeStr), "%H:%M", &timeinfo);
    }

    String ipStr = (WiFi.status() == WL_CONNECTED) ? WiFi.localIP().toString() : "No Wi-Fi";

    display.setCursor(0, 0);
    display.print(ipStr);
    display.setCursor(98, 0);
    display.print(timeStr);
    display.drawFastHLine(0, 10, 128, SSD1306_WHITE);

    display.setCursor(0, 14);

```

```

display.print("ST: ");
display.println(utf8rus(String(station)));
display.drawFastHLine(0, 32, 128, SSD1306_WHITE);

display.setCursor(0, 38);
display.println(utf8rus(String(track)));
display.display();
}

=== Файл: display_module.h ===
#pragma once
#include <Adafruit_SSD1306.h>

extern Adafruit_SSD1306 display;

void initDisplay();
void updateDisplay(const char* station, const char* track);
String utf8rus(String source);

=== Файл: fs_module.cpp ===
#include "fs_module.h"
#include <LittleFS.h>

void initFileSystem() {
    if (!LittleFS.begin(true)) {
        Serial.println("[КРИТ]: Ошибка монтирования LittleFS!");
        return;
    }
    Serial.println("[СИСТЕМА]: LittleFS успешно смонтирован.");
}

void removeFile(const char* path) {
    if (LittleFS.exists(path)) {
        LittleFS.remove(path);
    }
}

bool fileExists(const char* path) {
    return LittleFS.exists(path);
}

=== Файл: fs_module.h ===
#pragma once
#include <Arduino.h>

void initFileSystem();
void removeFile(const char* path);
bool fileExists(const char* path);

=== Файл: playlist_parser.cpp ===
#include "playlist_parser.h"
#include "../config.h"
#include "../global_state.h"
#include "system_logger.h"
#include <LittleFS.h>

void loadPlaylist() {
    stationCount = 0;
    if (!LittleFS.exists("/stations.txt")) {
        syslog("warn:", "ПАРСЕР", "Файл плейлиста /stations.txt отсутствует.");
        return;
    }

    File file = LittleFS.open("/stations.txt", "r");
    if (!file) {
        syslog("err:", "ПАРСЕР", "Не удалось прочитать /stations.txt.");
        return;
    }

    bool isFirstLine = true;

    while (file.available() && stationCount < MAX_STATIONS) {
        String line = file.readStringUntil('\n');
        line.replace("\r", "");

        // ОЧИСТКА СКРЫТОГО ВОМ (Byte Order Mark) ДЛЯ ПЕРВОЙ СТРОКИ
        if (isFirstLine) {
            isFirstLine = false;
            if (line.length() >= 3 && (unsigned char)line[0] == 0xEF && (unsigned char)line[1] == 0xBB && (unsigned char)line[2] == 0xBF) {
                line = line.substring(3); // Отрезаем ровно 3 мусорных байта
                syslog("info:", "ПАРСЕР", "Обнаружен и успешно удален скрытый UTF-8 ВОМ маркер.");
            }
            while (line.length() > 0 && (unsigned char)line[0] < 32) {
                line = line.substring(1);
            }
        }
    }
}

```

```

    line.trim();
    if (line.length() < 5 || line.startsWith("//")) continue;

    int separatorIdx = line.indexOf('\t');
    if (separatorIdx != -1) {
        stationList[stationCount].name = line.substring(0, separatorIdx);
        stationList[stationCount].url = line.substring(separatorIdx + 1);
        stationList[stationCount].name.trim();
        stationList[stationCount].url.trim();
        stationCount++;
    }
}
file.close();
sysLog("info:", "ПАРСЕР", "Загружено станций (разделитель таб): " + String(stationCount));

if (stationCount > 0) {
    File haFile = LittleFS.open("/ha_playlist.txt", "w");
    if (haFile) {
        for (int i = 0; i < stationCount; i++) {
            haFile.print(stationList[i].name);
            haFile.print('\t');
            haFile.println(stationList[i].url);
        }
        haFile.close();
        sysLog("info:", "ПАРСЕР", "Файл /ha_playlist.txt успешно сгенерирован.");
    }
}
}

=== Файл: playlist_parser.h ===
#pragma once

void loadPlaylist();

=== Файл: system_logger.cpp ===
#include "system_logger.h"
#include <LittleFS.h>
#include <WiFi.h>
// Если этому файлу нужны config или global_state, тоже подключаем через ../

// Физическое выделение буфера логгера в памяти (строго по доке Gyver)
sets::Logger webLogger(256);

String getResetReason(RESET_REASON reason) {
    switch (reason) {
        case POWERON_RESET: return "POWERON_RESET (Включение питания)";
        case RTCWDT_SYS_RESET: return "RTCWDT_SYS_RESET (Аппаратный Watchdog)";
        case DEEPSLEEP_RESET: return "DEEPSLEEP_RESET (Выход из глубокого сна)";
        case TGOBWDT_SYS_RESET: return "TGOBWDT_SYS_RESET (Сторожевой таймер Task WDT)";
        case RTC_SW_SYS_RESET: return "RTC_SW_SYS_RESET (Программный сброс системы)";
        case RTC_SW_CPU_RESET: return "RTC_SW_CPU_RESET (Программный сброс CPU)";
        default: return "UNKNOWN_RESET (" + String(reason) + ")";
    }
}

void syslog(const String& level, const String& module, const String& text) {
    String formattedLine = level + "[" + module + "]" + text;
    Serial.println(formattedLine);
    webLogger.println(formattedLine); // Логируем в ОЗУ веб-интерфейса

    // Дублирование лога в файл на LittleFS с защитой от переполнения
    if (LittleFS.begin(true)) {
        if (LittleFS.exists("/log.txt")) {
            File check = LittleFS.open("/log.txt", "r");
            if (check && check.size() > 30000) { // Ограничили 30 Кб для S2 Mini
                check.close();
                LittleFS.remove("/log.txt");
            } else if (check) {
                check.close();
            }
        }
        File file = LittleFS.open("/log.txt", "a");
        if (file) {
            file.println(formattedLine);
            file.close();
        }
    }
}

void logSystemHealth() {
    uint32_t freeHeap = ESP.getFreeHeap();
    uint32_t maxBlock = ESP.getMaxAllocHeap();
    long pingTime = -1;

```

```

    if (WiFi.status() == WL_CONNECTED) {
        IPAddress gateway = WiFi.gatewayIP();
        unsigned long start = millis();
        WiFiClient testClient;
        if (testClient.connect(gateway, 80, 100)) {
            pingTime = millis() - start;
            testClient.stop();
        }
    }

    String msg = "ОЗУ: " + String(freeHeap) + " Б (Блок: " + String(maxBlock) + " Б) | ";
    if (pingTime >= 0) {
        msg += "Пинг до шлюза: " + String(pingTime) + " мс | RSSI: " + String(WiFi.RSSI()) + " dBm";
    } else {
        msg += "Шлюз НЕ ОТВЕЧАЕТ! | RSSI: " + String(WiFi.RSSI()) + " dBm";
    }

    if (freeHeap < 40000 || (pingTime > 150 && pingTime != -1)) {
        syslog("warn:", "ДИАГНОСТИКА", msg);
    } else {
        syslog("info:", "ДИАГНОСТИКА", msg);
    }
}

bool checkUrlAvailability(const String& url) {
    if (WiFi.status() != WL_CONNECTED) return false;
    syslog("info:", "СЕТЬ", "Проверка доступности хоста...");

    WiFiClient client;
    String host = url;
    int port = 80;

    int protoIdx = host.indexOf("://");
    if (protoIdx != -1) host = host.substring(protoIdx + 3);

    int portIdx = host.indexOf(":");
    int pathIdx = host.indexOf("/");

    if (portIdx != -1 && (pathIdx == -1 || portIdx < pathIdx)) {
        port = host.substring(portIdx + 1, pathIdx).toInt();
        host = host.substring(0, portIdx);
    } else if (pathIdx != -1) {
        host = host.substring(0, pathIdx);
    }

    unsigned long start = millis();
    if (client.connect(host.c_str(), port, 2000)) {
        long duration = millis() - start;
        syslog("info:", "СЕТЬ", "Урл доступен. Время соединения: " + String(duration) + " мс");
        client.stop();
        return true;
    } else {
        syslog("err:", "СЕТЬ", "КРИТИЧЕСКАЯ ОШИБКА: Хост " + host + ":" + String(port) + " не отвечает!");
        return false;
    }
}

}

=== Файл: system_logger.h ===
#pragma once
#include <Arduino.h>
#include <rom/rtc.h>
#include <SettingsGyver.h>

// Даем другим модулям знать, что объект логгера существует
extern sets::Logger webLogger;

// Объявления функций диагностики
String getResetReason(RESET_REASON reason);
void syslog(const String& level, const String& module, const String& text);
void logSystemHealth();
bool checkUrlAvailability(const String& url);

```